Apollo3 Architecture Refactoring Report

2026-03-16

Contents

Apollo3 Refactoring Proposal

Colin Diesh

AI disclaimer: I used AI to generate this document. I manually reviewed most of the text in this document but it is AI generated so it could contain mistakes. I am not normally a fan of using AI

to write but I found it helpful to keep track of all the sweeping changes that were made. Claude Code was also extensively used during the refactoring.

Intro

This document describes a large proposal to make new features and improvements for Apollo 3. However, due to its size and change to the data model and schema, it could be seen as disruptive. Despite this, my hope is that this change will expand functionality, improve user experience, improve developer velocity, and help expand Apollo3 deployment options going forward.

Database migration (MongoDB to MikroORM). On origin/main, we use MongoDB to serve Apollo 3. This was driven by EMBL business needs. In the MongoDB data model, an entire gene is packed into a single MongoDB document. Now each feature and subfeature is given its own database row. This simplifies the server code significantly -- most edit operations go from 30+ lines of nested-document tree navigation down to a few targeted row updates. It also means the database itself enforces data relationships via foreign keys and cascade deletes, rather than relying on application code to keep things consistent (e.g. the manually maintained `allIds` arrays on every gene). See *From MongoDB to Relational Databases* for the full rationale and tradeoff analysis.

Simplified developer setup. On origin/main, the dev container configures a MongoDB replica set (required because MongoDB transactions only work with replica sets) and development requires 4 parallel processes. With this proposed refactoring, `pnpm install && pnpm start` is the complete setup -- it creates a SQLite database on the fly with no external dependencies.

Simplified production deployment. On origin/main, production deployment requires a MongoDB replica set (minimum two database containers, init scripts, extra Docker volumes, and an elevated transaction timeout setting). With this change, a single PostgreSQL container is sufficient, or SQLite for smaller deployments. Without analysis tools enabled, hosting could potentially run on a nano-sized instance. Analysis tools (BLAST, BLAT, miniprot, Tiberius) are more resource-intensive and would increase hosting requirements when enabled.

Desktop/Electron deployment. SQLite has no server process requirement, enabling fully self-contained desktop deployment -- something not possible with MongoDB.

Per-assembly storage and permissions. On origin/main, a single config.json holds every assembly and track and is served to all users with no access control. This limits scalability to large numbers of organisms. Now each assembly and track is its own record, and the server generates config.json per request filtered by user permissions. Assemblies can be public or private with per-user roles.

Multi-page application. On origin/main, all of Apollo3's UI (admin panels, organism management, user management) is packed into the JBrowse plugin itself. This makes the plugin heavy and makes it difficult to add new pages or workflows. This proposed refactoring moves to a multi-page Vite application where the JBrowse is one page among several. This allows lightweight, purpose-built pages.

Analysis tool integration. Added a generic server-side analysis framework with five built-in runners: local BLAST, NCBI remote BLAST, BLAT, miniprot, and Tiberius (deep learning gene prediction). Each tool has its own database type, job queue, and results renderer. New tools can be added by implementing a single runner interface and registering it -- no changes to the job or API layer are required.

Per-gene history tracking. On origin/main, Apollo3's change log is a global stream with no way to query the history of a single gene -- a capability Apollo2 had. This adds an indexed `geneId` column to the change log and a Recent Changes UI page with per-gene lookup.

Performance. 7-20x speedups across import, query, gff3 export, etc. The largest win (20x) came from quality checks that were re-running on every pan/zoom even when nothing had changed.

Security. Fixed 7 pre-existing vulnerabilities including an open redirect for OAuth token theft, missing cookie security flags, and a WebSocket CORS wildcard. Also significantly simplified the auth setup by not using InternetAccounts and instead leverages just 'normal' auth workflows using cookies and JWT.

Bug fixes. Found and fixed 3 pre-existing bugs on origin/main: check results being iterated as features, a refSeq delete that deleted the wrong scope, and a user location endpoint that sent garbled data on every update.

From MongoDB to Relational Databases

This branch migrates Apollo3 from MongoDB to MikroORM, supporting SQLite, PostgreSQL, and MongoDB from a single codebase. A migration script exists for existing MongoDB deployments.

At a Glance

Data model and editing

Area	MongoDB (current)	Relational (proposed)
Editing a single feature	Load entire gene doc, modify, write back	Update one row
Two users editing same gene	Second save may overwrite first user's changes	No conflict -- separate rows
Data integrity	Application-enforced	Database-enforced (FKs, cascades
Feature lookup by ID	Scan `allIds` arrays across all genes	Direct primary key lookup
Document/row size limits	16MB per gene document	None

Deployment and operations

Area	MongoDB (current)	Relational (proposed)
Desktop deployment	Not possible (MongoDB requires a server)	Fully self-contained with SQ
Server deployment	2 MongoDB containers in replica set, 4 volumes, init scripts	1 PostgreSQL container, or
Developer setup	Install + configure MongoDB replica set	`pnpm install && pnpm st`
CI setup	MongoDB service container + replica set init	Nothing needed (SQLite)
Hosting footprint	Replica set (multi-container)	Potentially a nano instance
Backups	`mongodump`	Copy one file (SQLite) or `pg`

Code and maintenance

Area	MongoDB (current)
Server code complexity	Each operation navigates nested trees; many are 30+ lines

Area	MongoDB (current)
Schema definitions	Mongoose schemas + separate `apollo-schemas` package
Dependencies	`mongoose`, `@nestjs/mongoose`, `connect-mongodb-session`, `mongoose-id-validator`,
Schema migrations	No formal migration system; schema changes are implicit
Real-time collaboration	Unchanged -- WebSockets, independent of DB
Undo/redo	Unchanged -- pure TypeScript, independent of DB

Why Migrate

1. **Every edit loads and rewrites the entire gene.** On origin/main, changing one exon by 1bp loads the full gene document (all exons, mRNAs, CDSs) and writes the whole thing back. With flat rows, it is a single-row update. This is the root cause of most code complexity on the server side -- each operation must navigate nested Maps, update parent bookkeeping, and serialize the whole tree back to the database.

2. **The `allIds` bookkeeping is fragile.** On origin/main, every gene carries a manually-maintained array of all descendant IDs. Application code must keep this array in sync on every add, delete, or reparent. With flat rows, every feature has its own primary key -- no bookkeeping needed.

3. **Concurrent editing.** Because the full gene document is loaded and saved as a unit, two annotators editing different exons of the same gene both load and save the full document -- the second save could overwrite the first user's changes. This is a known limitation of the document-per-gene model (addressable with optimistic locking in MongoDB, but not currently implemented on origin/main). With flat rows, edits to different features target different rows and do not conflict. A database-level counter with pessimistic locking assigns each change a unique sequence number within a transaction, preventing ordering collisions.

4. **16MB document size limit.** Highly spliced genes can hit MongoDB's per-document ceiling. Flat rows have no per-record limit.

5. **Replica set required for transactions.** MongoDB requires a replica set (minimum two containers) to support transactions. On origin/main, Apollo3 uses transactions for multi-step edits. A single-node MongoDB deployment does not support transactions. The replica set also requires extra Docker volumes, an init script, and an elevated timeout setting (`transactionLifetimeLimitSeconds=300`) for large imports. PostgreSQL needs one container; SQLite needs none -- both support transactions natively.

6. **No desktop/Electron deployment.** MongoDB requires a running server process with no embedded mode. SQLite enables fully self-contained desktop/Electron deployment.

7. **No formal schema migration system.** On origin/main, there is no built-in way to version or migrate schema changes -- changes are implicit (start writing new fields and hope old documents still work). MikroORM provides timestamped migration files committed to the repo, run in order on deploy, and reversible. While NoSQL schemas are more flexible by nature (no columns to add), that flexibility comes at the cost of no enforcement and no audit trail. Automated migrations give us schema flexibility with a safety net.

Why MikroORM

- Multi-database from one codebase (SQLite, PostgreSQL, MongoDB, MySQL, MS SQL)
- TypeScript-first entity definitions with compile-time type safety
- Unit of Work pattern: automatic change tracking, single-transaction flush
- Built-in migration system (like Liquibase/Rails migrations)
- First-class NestJS integration via `@mikro-orm/nestjs`

Tradeoffs: What Gets Harder

MongoDB's nested document model has genuine advantages for certain operations:

Operation	MongoDB advantage
Loading a full gene tree	One document fetch returns the entire gene pre-assembled
Merging transcripts	All children already in memory from the document load
Attribute search	MongoDB natively queries inside nested documents and supports `$text` indexes with r
Schema flexibility	Adding a field requires no migration -- just start writing it

These tradeoffs are real. The operations that get harder (tree loading, transcript merges, attribute search) are less frequent than those that get dramatically simpler (single-feature edits, feature lookup, bulk import, deletion), but the attribute search limitation in particular deserves attention as the annotation workflow matures.

New Deployment Options

The relational model enables deployment patterns not practical with MongoDB:

- **Nano instance with SQLite**: full Apollo3 in one process on minimal hardware, when analysis tools are not in use. Analysis tools (BLAST, BLAT, miniprot, Tiberius) are more resource-intensive and would increase requirements.
- **Scale-to-zero containers** (Fargate/Cloud Run + Aurora Serverless/Neon): no idle cost when nobody is using the instance
- **Desktop/Electron**: SQLite has no server process requirement, enabling fully self-contained desktop deployment -- not possible with MongoDB
- **Lambda** (read-only): viable for serving annotations to public JBrowse; full collaboration blocked by WebSocket requirement (solvable with SSE)

Schema Comparison: MongoDB vs Relational

The schema has the same entities. The key change is **how features are stored** and **how relationships are enforced**.

The One Big Change: Feature Storage

MongoDB: nested documents

```
features collection:

  _id: gene_1
```

```
type: "gene"
allIds: [gene_1, mRNA_1, exon_1, exon_2, ...]  <- manual bookkeeping
children: {
  mRNA_1: {
    children: {
      exon_1: { type: "exon", min: 1000 ... }
      exon_2: { type: "exon", min: 3000 ... }
      CDS_1:  { type: "CDS",  min: 1200 ... }
    }
  }
}
```

One record per gene. Every edit loads and saves the whole document. `allIds` must be manually synced. 16MB document size limit.

Relational: flat rows with parent references

```
feature table:
+--------------+--------------+------------+-------+-------+
| _id          | parent       | type       | min   | max   |
+--------------+--------------+------------+-------+-------+
| gene_1       | NULL         | gene       | 1000  | 5000  |
| mRNA_1       | gene_1       | mRNA       | 1000  | 5000  |
| exon_1       | mRNA_1       | exon       | 1000  | 1500  |
| exon_2       | mRNA_1       | exon       | 3000  | 3500  |
| CDS_1        | mRNA_1       | CDS        | 1200  | 3400  |
+--------------+--------------+------------+-------+-------+
```

One row per feature. Editing exon_2 touches only that row. No `allIds`. No size limit. `ON DELETE CASCADE` on parent FK handles child cleanup.

Practical comparison

Scenario	MongoDB	Relational
Edit one exon	Load entire gene, modify, write back	Update one row
Two users edit different exons	Second save may overwrite first user's changes	No conflict -- separate rows
Look up feature by ID	Scan `allIds` arrays	Primary key lookup
Delete a gene	One delete (whole doc)	One delete (CASCADE remo
Large gene (thousands of exons)	May hit 16MB limit	No limit
Add/remove child	Update parent's `allIds` + save	Insert/delete one row

Relationship Enforcement

MongoDB relies on application code. The relational schema uses foreign keys with CASCADE delete -- the database enforces relationships automatically.

```
AssemblyEntity
  - RefSeqEntity (FK -> assembly, CASCADE)
    - FeatureEntity (FK -> refSeq, CASCADE; FK -> parent, CASCADE)
```

```
    - RefSeqChunkEntity (FK -> refSeq, CASCADE)
    - CheckResultEntity (FK -> refSeq, CASCADE)
  - ExportEntity (FK -> assembly, CASCADE)
```

Deleting an assembly: one `DELETE` statement. The database removes all refSeqs, features, chunks, check results, and exports automatically.

Unaffected Systems

- Real-time collaboration (WebSockets)
- Undo/redo (TypeScript change classes)
- Change tracking (same audit log structure)
- User management, file handling

Performance Fix Discovered During Migration

On origin/main, `GET /features/getFeatures` calls `checksService.checkFeature()` on every feature in the returned range. For each feature and each enabled check, this deletes all existing check results from the database, re-executes the check logic (which may involve sequence lookups), and inserts the newly computed results back. There is a timestamp guard that skips checks whose definition has not changed since the feature was last modified, but in practice most checks still run. For a viewport containing 1000 genes, every pan or zoom triggers thousands of DELETE + compute + INSERT cycles. This change moves check execution to the mutation pipeline so checks run only after edits, and `GET` returns pre-computed results. Result: **20x speedup** (74s -> 3.65s for 1000 genes).

Code citations (origin/main)

features.service.ts -- `findByRange()` calls `checkFeature()` in a loop on every returned feature:

```
async findByRange(searchDto: FeatureRangeSearchDto) {
  const featureDocs = await this.operationsService
    .executeOperation<GetFeaturesOperation>({ ... })
  for (const featureDoc of featureDocs) {
    await this.checksService.checkFeature(featureDoc)
  }
  const checkResults = await this.checksService.findByRange(searchDto)
  return [featureDocs, checkResults]
}
```

checks.service.ts -- `checkFeature()` deletes and re-inserts results per feature per check:

```
async checkFeature(doc: FeatureDocument, checkTimestamps = true) {
  const checks = await this.getChecksForAssembly(doc)
  for (const check of checks) {
    if (checkTimestamps && doc.updatedAt && check.updatedAt < doc.updatedAt) {
      continue
    }
    await this.clearChecksForFeature(doc, check.name)   // DELETE
    const c = checkRegistry.getCheck(check.name)
```

```
    const result = await c.checkFeature(flatDoc, ...)    // COMPUTE
    if (result.length > 0) {
      await this.checkResultModel.insertMany(result)     // INSERT
    }
  }
}
```

Technical Details: Relational Migration

Worked examples, tradeoff analysis, and schema assessment.

Worked Example: Changing an Exon Boundary

When an annotator moves an exon boundary outward, up to three rows may change: the exon, the transcript (if exon extends beyond it), and the gene.

- **MongoDB**: Scan `allIds` -> load entire gene document (all 50 exons to change 1) -> walk nested structure -> modify one field -> write everything back
- **Relational**: Three targeted single-row updates. Nothing else read or written. No risk of overwriting a concurrent edit to a different exon.

Open question: Both models rely on the client to send correct parent boundary updates. A server-side parent boundary verification after child coordinate changes would make the system defensively correct.

Where Relational Is Harder

Gene tree loading for range queries

The most common read: "get all features overlapping this viewport."

- **MongoDB**: One query returns complete nested gene trees.
- **Relational**: Two steps: (1) find root features by range (indexed, fast), (2) load all descendants via recursive CTE.

`findDescendantsOfMany` already batches all roots into one CTE query. A `root_id` denormalization column has been proposed but introduces a sync burden -- **defer unless profiling proves this is a bottleneck**.

Text search

Current `LIKE`-based search is weaker than MongoDB's text indexes. SQLite FTS5 and PostgreSQL tsvector are both mature built-in replacements requiring no architectural changes.

Tradeoff summary

Operation	Status	Notes
Gene tree loading	Acceptable	Recursive CTEs batch efficiently
Gene/assembly deletion	**Fixed**	`ON DELETE CASCADE` on all FKs

Operation	Status	Notes
Bulk queries (search, export)	**Fixed**	Batched `IN` filters
N+1 query patterns	**Fixed**	Level-batched BFS, in-memory parent maps
Feature counting	**Fixed**	`em.count()` instead of load-all
Full-text search	Fixable	FTS5 / tsvector
Single-feature edits	Faster	One row vs full document
Concurrent edits	Safer	Separate rows, no contention
Large imports	Better	Streaming, transactions, no size limit

Schema Assessment

What is solid

- One row per feature with parent FK, coordinate columns, composite index on `(refSeq, min, max)`
- 13 entities in ~330 lines, each mapping to a table with typed columns
- Clean separation: entities define structure, repositories handle queries, change classes contain business logic

Implemented improvements

1. **CASCADE deletes** on all FKs -- assembly deletion is a single `deleteById`
2. **Index on `FeatureEntity.parent`** -- tree traversal uses indexed queries
3. **Index on `RefSeqEntity.assembly`** -- assembly lookups indexed
4. **Index on `CheckResultEntity.name`** -- check filtering indexed
5. **Batched descendant queries** -- BFS with `IN` filters (D queries per tree depth, typically 3-4, vs N queries per node)
6. **In-memory parent map for search** -- eliminates per-match DB queries

Remaining improvements

- **(Deferred) `root_id` column** -- single-query tree loading, but requires sync on reparent. Only if profiling justifies it.

Schema relationships

```
AssemblyEntity
  `--- RefSeqEntity (FK: assembly)
       |--- FeatureEntity (FK: refSeq; FK: parent -> self)
       |--- RefSeqChunkEntity (FK: refSeq)
       `--- CheckResultEntity (FK: refSeq)

ChangeEntity (FK: reverts -> self, for undo chain)
ExportEntity (FK: assembly)
UserEntity, FileEntity, CheckEntity, JBrowseConfigEntity, CounterEntity (standalone)
```

All FKs have CASCADE delete.

Schema Migrations

MongoDB has no formal migration system. MikroORM provides timestamped migration files (like Liquibase/Rails) that are committed to the repo, run in order on deploy, and can be rolled back.

Currently using `SchemaGenerator.updateSchema()` at startup (appropriate for development). Production should transition to explicit committed migration files for auditability.

Multi-Database Repository Factory

`DB_BACKEND`	Feature Repository	Tree Strategy
`sqlite` (default)	`MikroOrmFeatureRepository`	Recursive CTEs
`postgresql`	`MikroOrmFeatureRepository`	Recursive CTEs
`mongo`	`MongoFeatureRepository`	Iterative BFS via generic EntityManager

All other repositories use `MikroOrm*Repository` implementations with the generic `EntityManager` API (works with any driver).

Alternatives to the MikroORM Migration

Evaluation of other paths considered.

Option 1: Stay on MongoDB With Targeted Fixes

Phase 1 -- Eliminate `allIds` bottleneck (1-2 weeks)

Replace the `allIds` array scan with a secondary index or lookup collection mapping feature IDs to their parent document.

Phase 2 -- Add concurrency protection (2-3 weeks)

Optimistic locking (version field) or pessimistic locking to prevent one annotator's save from overwriting another's. Doesn't eliminate the root issue (full document load/save) but prevents silent data loss.

Phase 3 -- Partial document flattening (4-6 weeks)

Split at the transcript level: each transcript becomes its own document (exons still nested inside). Reduces blast radius -- editing exon_1 no longer loads unrelated mRNA_2.

Remaining problems: Two annotators editing different exons in the *same* transcript still conflict. No Electron/desktop support. Replica set still required. `allIds` still needed at the transcript level.

Assessment

Comparable effort to the relational migration. Addresses some concurrency issues but not fully (same-transcript conflicts remain). Does not address desktop deployment or simplify the operational setup.

Option 2: Firestore / Firebase

Why tempting: Built-in auth (Google, Microsoft, email), serverless Cloud Functions, real-time sync, zero infrastructure. Would replace ~15 files of Passport/JWT/session code.

Why problematic for Apollo3:

Issue	Impact
No MikroORM driver	Would need to replace the ORM entirely, losing SQLite/PostgreSQL portability
Vendor lock-in	Proprietary to Google Cloud; no standard SQL; pricing subject to change
No offline/Electron	Requires network to Google servers -- same hard constraint as MongoDB
No WebSocket support	Cloud Functions don't support persistent connections; would need to rearchitect collabo
Still document-based	No FKs, no cascades, no joins, no standard query language

Viable hybrid: Use Firebase Authentication (standalone) while keeping MikroORM for data. This captures the auth simplification without database lock-in. Compatible with the relational migration as an independent improvement.

Option 3: Parallel Backend Implementations

The repository interface pattern technically allows multiple backend implementations. In practice, maintaining two complete repository sets, test suites, and deployment configs roughly doubles maintenance burden. The MikroORM migration period (when both MongoDB and relational paths coexisted) confirmed this cost.

Only worthwhile if two fundamentally different deployment targets are needed. The relational model already covers desktop (SQLite) through production server (PostgreSQL).

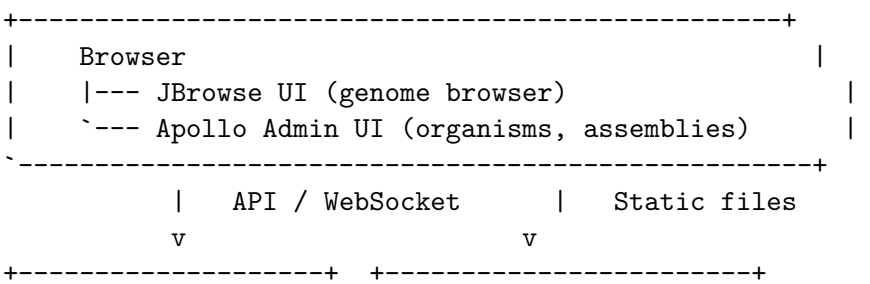
Recommendation

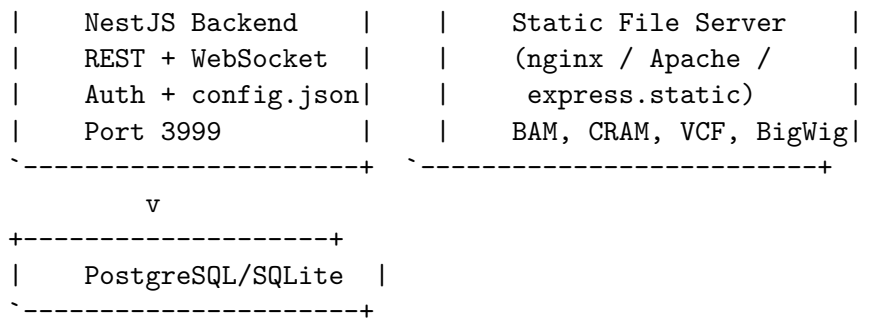
The relational migration addresses desktop deployment, operational complexity, data model challenges, and hosting cost in one coherent change. If not preferred, a pragmatic fallback:

1. Targeted MongoDB fixes (Phases 1-2) for near-term stability
2. Firebase Auth as standalone service for auth simplification
3. Revisit relational migration when desktop/Electron becomes a priority

Deployment Guide

Architecture





Deployment Options

Option 1: Single Server (simplest)

NestJS serves everything. Set `JBROWSE_STATIC_DIR` to the JBrowse web directory. Good for development and small teams (< 10 users).

Option 2: nginx + NestJS (recommended for production)

nginx serves static files with zero-copy `sendfile()`. NestJS handles API and WebSocket only. See `deploy/nginx/`.

```
cd deploy/nginx && cp .env.example .env && docker compose up -d
```

Option 3: Apache httpd + NestJS

Same role as nginx. See `.github/workflows/deploy/` for Apache configuration.

Static file performance

Server	Mechanism	Notes
nginx	`sendfile()` -- zero-copy	Multi-process, excellent throughput
Apache	`sendfile()` via `mod_mpm_event`	Multi-threaded, excellent throughput
Node.js	`fs.createReadStream()` with byte offsets	Single-threaded, adequate for small teams

Use nginx/Apache when: BAM/CRAM > 1GB, 10+ concurrent users, or production.

nginx Routing

```
nginx
  |--- /jbrowse/config.json -> proxy to NestJS (dynamic, generated from DB)
  |--- /config.json          -> proxy to NestJS
  |--- /socket.io/           -> proxy to NestJS (WebSocket upgrade)
  |--- /jbrowse/*.bam|cram|bw|html|js -> serve from disk (sendfile)
  |--- /files/<checksum>     -> serve from uploads dir
  `--- everything else       -> proxy to NestJS (API)
```

`JBROWSE_STATIC_DIR` is **not set** on NestJS when using nginx. `config.json` is always proxied (dynamically generated from track/assembly records in DB).

Database

Backend	Use case	Config
SQLite	Dev, desktop, small deployments	Default (no config)
PostgreSQL	Production collaborative	`DB_BACKEND=postgresql DB_CONNECTION_URL`
MongoDB	Existing deployments migrating from origin/main	`DB_BACKEND=mongo DB_CONNECTION_URL=mong`

Environment Variables

Required

Variable	Description
`URL`	Public URL (e.g. `https://apollo.example.com`)
`NAME`	Instance name shown in UI
`FILE_UPLOAD_FOLDER`	Directory for uploaded files
`PORT`	Server port (default: 3999)

Secrets

Variable	Description
`JWT_SECRET`	JWT signing secret (min 32 chars)
`SESSION_SECRET`	Session cookie secret (min 32 chars)

If not set, the server auto-generates random secrets and persists them to `dist/.secrets/`. Survives restarts. For production, set explicitly or use `_FILE` variants (e.g. `JWT_SECRET_FILE=/run/secrets/jwt`) for Docker secrets.

Optional

Variable	Description
`JBROWSE_STATIC_DIR`	JBrowse static files (single-server only)
`DB_BACKEND`	`sqlite` / `postgresql` / `mongo`
`DB_CONNECTION_URL`	Database connection string
`ALLOW_GUEST_USER`	Allow unauthenticated guest (default: false)
`GUEST_USER_ROLE`	Guest role: `admin` / `user` / `readOnly`
`DEFAULT_NEW_USER_ROLE`	New user role: `admin` / `user` / `readOnly` / `none`
`GOOGLE_CLIENT_ID` / `_SECRET`	Google OAuth
`MICROSOFT_CLIENT_ID` / `_SECRET`	Microsoft OAuth
`ALLOW_ROOT_USER`	Enable root password login
`ROOT_USER_PASSWORD`	Root admin password

First-Time Setup

On first start with no admin, the server prints a one-time setup URL:

`Setup URL: http://localhost:3999/auth/setup?token=<random-token>`

Visit it, then log in (Google, Microsoft, or root). That account becomes admin. Token is single-use. For dev, `GUEST_USER_ROLE=admin` skips this.

Architecture Overview

Per-Assembly Storage

On origin/main, the server maintains a single JBrowse `config.json` document in the database that describes every assembly, every evidence track, and every search adapter for the entire Apollo3 instance. When an admin adds a track or modifies an assembly, the server rewrites this entire document. All users see the same configuration -- there is no mechanism to show different assemblies or tracks to different users.

This proposal replaces it with a normalized data model where each assembly, evidence track (BAM, VCF, BigWig, CRAM), BLAST database, and text search adapter is stored as its own database record. Tracks and BLAST databases are linked to assemblies through many-to-many relationships, so a single track can appear on multiple assemblies. The server now generates `config.json` dynamically for each request, including only the assemblies and tracks that the requesting user has permission to see.

This means teams can manage their own evidence tracks independently -- uploading, modifying, or removing tracks on their assemblies without admin intervention and without affecting other assemblies on the same instance.

Per-Assembly Permissions

Access control operates at two levels. Global roles (`none`, `readOnly`, `user`, `admin`) set a baseline for what a user can do across the instance. On top of this, admins can assign per-assembly roles that override the global default for specific assemblies. Each assembly also has a visibility setting -- public assemblies are visible as read-only to all authenticated users, while private assemblies are visible only to users with an explicit permission grant.

When a user accesses an assembly, the system resolves their effective role by checking (in order): global admin status, then per-assembly permission, then assembly visibility. This allows scenarios like a postdoc having edit access to their own genome, read-only access to a collaborator's public assembly, and no access to another lab's private data -- all on the same server.

Summary of Changes

Change	Current (origin/main)	Proposed
Assembly/track storage	Single monolithic config document	Individual records with many-to-many relation
Access control	All users see everything	Per-assembly roles with public/private visibilit
Analysis tools	External tools, manual result transfer	Generic runner framework: local BLAST, NCB
Database	MongoDB (replica set required)	SQLite, PostgreSQL, or MongoDB via single co

Performance Optimization Report

7-20x speedups across all major operations on a 5,000-feature synthetic dataset.

Results

Operation	origin/main	Proposed	Speedup
Assembly import (5000 features)	60s	8.06s	**7.5x**
Feature get (all)	74s	3.65s	**20x**
Feature search	4.5s	3.41s	1.3x
GFF3 export	6.3s	3.86s	1.6x
Assembly delete	5.0s	3.46s	1.4x

Remaining ~3-4s baseline is CLI startup + HTTP overhead, not DB operations.

What Changed

1. SQLite WAL mode + synchronous tuning

`PRAGMA journal_mode = WAL` + `PRAGMA synchronous = NORMAL` at startup. Concurrent reads during writes, reduced fsync for batch inserts.

2. Raw SQL for read-only queries

Replaced `em.find()` with `em.getConnection().execute()` for all read-only feature queries. ORM hydration (proxy creation, identity map, change tracking) is pure overhead since results are immediately converted to plain objects.

3. Recursive CTEs for tree operations

On origin/main, iterative BFS issues N queries per depth level per root. Replaced with single recursive CTE queries for `findDescendantsOfMany`, `deleteDescendants`, and `findRootParent`. For 5000 features across ~1000 gene trees: hundreds of queries -> 1.

4. Moved check recalculation from GET to mutation pipeline

The single largest performance issue -- responsible for the 20x speedup.

On origin/main, `GET /features/getFeatures` re-runs all quality checks on every root feature in the response. For 1000 genes, each pan/zoom triggers: 1000x findById + 1000x findDescendants + 1000x assembleFeatureTrees + check config lookups + delete/rerun/save checks. All redundant -- results are already persisted from the last edit.

Fix: checks run after mutations only. GET returns pre-computed results.

Benchmark Environment

```
Dataset: Synthetic (1000 genes, ~5000 features), 3 iterations
Node.js v24.13.0, linux x64, 2026-03-14
```

Reproduction

```
cd packages/apollo-cli && pnpm tsx src/test/benchmark.ts --synthetic
```

Authentication & Security Audit

Two rounds of audits covering authentication, controllers, data handling, and injection surfaces on origin/main. All critical/high issues addressed in this branch.

Issues Found and Fixed

#	Severity	Issue
1	CRITICAL	**Open redirect** -- OAuth callback accepts arbitrary `redirect_uri`, allowing token theft via `r`
2	CRITICAL	**Missing Secure flag** -- session cookie set without `secure: true`
3	HIGH	**WebSocket CORS wildcard** -- `cors: { origin: '*' }` on WebSocket gateway
4	HIGH	**JWT logged in plaintext** -- full token logged at DEBUG level
5	BUG	**OAuth client ID file-read** -- file contents overwritten by file path (`microsoftClientID = c`
6	MEDIUM	**Session cookies lacked security options** -- no `httpOnly`, `secure`, `sameSite`, `maxAge`
7	MEDIUM	**No minimum secret length** -- single-char secrets accepted

Code citations (origin/main)

Open redirect -- `authentication.controller.ts` passes the `redirect_uri` query parameter directly through to the OAuth flow without validating it against the server's configured URL:

```
// authentication.controller.ts -- handleLogin()
const url = redirect_uri
  ? `${type}?${new URLSearchParams({ redirect_uri }).toString()}`
```

WebSocket CORS wildcard -- `messages.gateway.ts` accepts connections from any origin:

```
// messages.gateway.ts
@WebSocketGateway({ cors: { origin: '*' } })
```

Session cookie without Secure flag -- `main.ts` configures express-session without cookie security options:

```
// main.ts
app.use(session({
  secret: sessionSecret,
  resave: false,
  saveUninitialized: false,
  store: new MongoDBStore({ uri: mongodbURI, collection: 'expressSessions' }),
  // no cookie options -- defaults to insecure
}))
```

JWT logged in plaintext -- `authentication.service.ts` logs the full token:

```
// authentication.service.ts
this.logger.debug(
  `First time login successful. Apollo token: ${JSON.stringify(returnToken)}`,
)
```

Verified Secure

- All 14 controllers have class-level auth decorators; default is `Role.Admin`
- SQL queries use MikroORM parameterized queries (no injection risk)
- File uploads store by checksum, not user-provided filename (no path traversal)
- Root password comparison uses plaintext `===` against env var (intentional -- hashing env vars provides no security since attacker with env access already has the password)

Accepted Risks

- Token in OAuth redirect URL: mitigated by origin validation + short-lived popup
- No CSRF middleware: mitigated by `SameSite=lax`
- No refresh token: 24-hour JWT expiry is acceptable
- No token revocation on logout: standard for stateless JWT

Bug Fixes, Simplifications & Code Quality

Bugs Fixed

1. updateChecks iterating check results as features

`assemblies.service.ts` -- on origin/main, `findByRange()` returns `[features, checkResults]`. The code iterates both arrays, passing check result IDs to `checkFeature()`. **Fix**: Split into `findFeaturesByRange` (features only). Check results fetched separately via `GET /checks/range`.

2. RefSeqsService.remove() deletes wrong scope (dead code)

On origin/main, this looks up one refSeq by ID, then calls `deleteByAssembly()` which deletes ALL refSeqs for the assembly. **Fix**: Simplified to accept `assemblyId` directly.

3. User location endpoint encoding bug

On origin/main, frontend sends `URLSearchParams(JSON.stringify(locations))` -- garbled data, 500 errors on every location update. **Fix**: Proper JSON with `Content-Type: application/json`. Also simplified from array of all visible regions to single primary region (users view one region at a time).

Architectural Simplifications

Developer setup

On origin/main, development requires 4 parallel processes (shared watch + NestJS

- plugin dev server + sibling jbrowse-components clone via justfile). This change reduces it to a single NestJS server on port 3999. Removes `npm-run-all`, `concurrently`, `serve`, justfile. Setup becomes `pnpm install && pnpm start`.

WebSocket channels

On origin/main, WebSocket uses per-refSeq channels (`${assemblyId}-${refSeqName}`). This change consolidates to a single `COMMON` channel. Removes 12 lines of DB queries per change (feature->refSeq->name lookups), `ensureAssemblySocket()` (25 lines), `haveDataForChange()` (12 lines). Safe because annotation edit volume is human-speed. Channel name constants extracted to shared `Messages.ts`.

InternetAccount removal

Removes the `ApolloInternetAccount` JBrowse abstraction (~500 lines, 6 files). Cookie auth makes it redundant -- `credentials: 'same-origin'` handles everything. WebSocket and change tracking move to session model. `baseURL`, `role`, `userId` now come from `ApolloPlugin` config instead of JWT decode.

Code Simplification

Change	Current	Proposed
API separation	`getFeatures` returns `[features, checkResults]` tuple	Separate endpoints, pa
Export service	111-line monolith	Focused helpers (`write`
GFF3 export	Duplicate hierarchy assembly (~40 lines)	Shared `assembleFeatu`
ServerDataStore	Anonymous function wrappers around `filesService` methods	Direct `filesService` r
findByFeatureIds	Set-based dedup after `SELECT DISTINCT`	DB handles it
RefSeqsService.update	12-line manual property mapping	Spread operator (4 line
Dev Container	MongoDB extension + `mongosh` + port 27017	PostgreSQL + port 543

Analysis Tool Integration

Apollo3 includes a generic server-side analysis framework (`/analysis/`) where each tool is an independent runner behind a shared job queue and REST API.

API surface

Three endpoint groups cover the full workflow:

- `GET /analysis/tools` -- list available tools and their parameter schemas
- `GET/POST/DELETE /analysis/databases` -- CRUD for tool-specific databases (BLAST indexes, BLAT databases, etc.) and triggering a build job
- `GET/POST /analysis/jobs` -- submit a job, poll status, cancel, and retrieve results

Built-in runners

Five runners ship by default:

Runner	Description
`local-blast`	blastn/blastp/blastx/tblastn/tblastx against a locally built BLAST database
`ncbi-blast`	Same programs forwarded to NCBI's remote BLAST servers

Runner	Description
`blat`	Fast nucleotide/protein alignment against a local BLAT database
`miniprot`	Protein-to-genome alignment via miniprot
`tiberius`	Gene prediction via Tiberius deep learning model

Each runner implements the `AnalysisRunner` interface (submit, poll, cancel, parse results). Jobs and databases are stored as `AnalysisJobEntity` and `AnalysisDbEntity` rows with a `tool` field and flexible `params`/`results` JSON columns, so no schema changes are needed when adding a new tool.

Frontend

A tabbed *Sequence Search* page shows one tab per available tool. Each tab has its own form and results renderer. Admins have a separate *Jobs* panel for building databases.

Adding a new tool

Adding a new tool requires four steps:

1. Create `runners/my-tool.runner.ts` implementing `AnalysisRunner`
2. Register it in `analysis.module.ts`
3. Inject it in the worker and service constructors
4. Add a results renderer in `sequence-search.tsx` if the output format differs from existing tools

CheckResult Data Model Simplification

Change

Replaces `ids: string[]` JSON array on `CheckResultEntity` with a single indexed `featureId: string` column.

Why

On origin/main, both existing check implementations produce exactly one feature ID per result. The array is designed for a multi-feature check scenario that has not been needed. If that scenario arises, a junction table would be a better fit than a JSON array.

The `LIKE`-based query on the JSON blob on origin/main has a false-positive matching bug (`"feat-1"` can match `"feat-123"`), requires a two-stage query+filter pattern, and prevents indexing.

Impact

Aspect	Current (origin/main)	Proposed
Query	`LIKE '%"id"%'` + in-memory filter (full table scan)	Indexed `WHERE featureId = ?` (O(log n))
Entity	`ids: p.json<string[]>()`	`featureId: p.string()` + index
MST	`types.array(types.safeReference(...))`	`types.safeReference(...)`
Repository	121 lines, 17-line two-stage delete	96 lines, 3-line direct delete

Aspect	Current (origin/main)	Proposed

Also Done: Repository Instance Caching

On origin/main, `DatabaseService` creates new repository instances on every getter call. This change creates them once in the constructor and reuses them -- eliminates ~12 object allocations per HTTP request. Safe because `RequestContext` middleware provides per-request EM isolation via `AsyncLocalStorage`.

InternetAccount Removal

Change

Removes the `ApolloInternetAccount` JBrowse plugin abstraction (~500 lines, 6 files). Replaces it with direct cookie-based auth and session-level connection management.

Why

With cookie-based auth (HTTP-only cookies), the InternetAccount becomes redundant:

- `CollaborationServerDriver.fetch()` already uses `credentials: 'same-origin'`
- Cookie auth is handled transparently by the browser
- Multi-account selection UI adds complexity for a feature no deployment uses (Apollo3 is single-server)

What Moved Where

Concern	Current (origin/main)	Proposed
WebSocket management	`ApolloInternetAccount/model.ts`	Session model (`session.ts`)
Change sequence tracking	`ApolloInternetAccount/model.ts`	Session model
`baseURL`, `role`, `userId`	JWT token decode + `internetAccounts` config	`ApolloPlugin` config in `config.j`
API calls	`internetAccount.getFetcher()`	`apolloFetch()` with `credentials`
Multi-account UI	~200 lines across 6 components	Deleted

Login Flow (Proposed)

1. Plugin reads `baseURL` from `ApolloPlugin` configuration
2. Session fetches `${baseURL}/jbrowse/config.json` with cookie credentials
3. Server returns config with `role`, `userId`, and assemblies (if authenticated)
4. Session initializes WebSocket and admin menus based on role

Impact

Metric	Current (origin/main)	Proposed
InternetAccount files	6	0
Auth mechanisms	Cookie + JWT + Authorization header	Cookie only

Metric	Current (origin/main)	Proposed
Plugin bundle	1.57 MB	1.56 MB

Technical Notes

Deep technical analysis of the flat-row data model, migration cleanup, and remaining improvement opportunities.

Flat Rows vs Nested Documents: Operation-by-Operation

Single-feature edits (most common)

All single-feature operations (change coordinates, type, strand, attributes) follow the same pattern:

- **MongoDB**: Load entire gene doc -> walk tree -> modify -> save whole doc (~70 lines)
- **Flat rows**: Look up one row -> update -> done (2 queries, ~15 lines)

Flat rows produce shorter code, fewer database reads/writes, and less risk of unrelated data being touched. The tradeoff is that features are now on separate rows, so operations that need the full gene tree (like validation checks) require an extra assembly step -- a recursive query to collect descendants. For single edits this cost doesn't apply.

Adding features

Operation	MongoDB	Flat rows
New top-level gene	Create one nested doc	Flatten sn
Add exon to existing mRNA	Load gene, insert child, update `allIds`, save whole doc	Insert one
GFF3 import	One doc at a time, no transactions (16MB limit), OOM on large files	Flatten to

Deleting features

Operation	MongoDB	Flat rows
Delete top-level gene	Delete one document	`ON DELETE CASCADE` han
Delete exon from mRNA	Load gene, find exon in tree, remove, update `allIds`, save	Delete one row

Structural edits

Operation	MongoDB	Flat rows
Split exon	Load gene, create two children, update `allIds`, save	Insert two rows, delete old row
Merge exons	Load gene, merge in memory, save	Update first exon bounds, delete secon
Merge transcripts	All in-memory on one document, single save	Query children separately, reparent, d

Merging transcripts is the clearest case where nested documents have an advantage -- all children are already in memory. With flat rows, this requires separate queries and reparenting, though batching can reduce the overhead.

Undo operations

Delete what forward created, re-insert what forward deleted. No tree navigation or `allIds` bookkeeping. Simpler in the flat model.

Read operations

Operation	MongoDB	Flat rows
Features in coordinate range	Returns full nested trees (loads more than needed)	Returns exactly matching
Find feature by ID	`findOne({allIds: id})` + tree walk	Primary key lookup
Find all CDS on a chromosome	Load all genes, walk all trees	`WHERE type='CDS' AND r`
Count features by type	Load all, count in app code	`GROUP BY type`
Export to GFF3	Data already nested	Rows map to GFF3 lines
Run validation checks	Data already nested	Fetch descendants + asser
Text search on attributes	`$text` index (ranked, stemmed)	`LIKE` on JSON column (n

For targeted queries (by ID, by type, by range), the relational model benefits from standard database indexing. For operations that need the full gene tree (validation, client display), there is an extra step to collect and reassemble descendants. MongoDB returns these pre-assembled, which is convenient. The cost of reassembly is modest (one recursive CTE query + O(n) in-memory pass), but it is a real tradeoff.

Operations MongoDB does not do well

- **Reparent a feature**: Flat rows: `UPDATE SET parent = :new WHERE _id = :id`. MongoDB: load both source/destination gene docs, move between nested Maps, update both `allIds`, save both docs.
- **Query across hierarchy**: "genes with exons < 50bp?" -- a WHERE clause with flat rows. MongoDB: load all gene docs, walk every tree.
- **Stream imports**: Flat rows stream to DB with bounded memory. MongoDB requires building nested trees in memory first; OOM on large files.

Summary

Simple edits (the majority of annotation work) are shorter code, faster, and touch less data with flat rows. Complex structural edits like transcript merges require more database round-trips, though this is addressable with batch queries. Read operations gain precise indexed queries at the cost of a tree-assembly step when the full hierarchy is needed. The nested document model avoids that assembly step, but at the cost of loading and rewriting entire genes for every operation -- including simple ones.

Migration Cleanup

Unified execution path

Removes the dual `executeOnServer()` / `executeOnServerV2()` implementations from all 23 Change classes and 2 Operation classes. Removes `ServerDataStoreV2`. Single code path.

Eliminated FeatureChange helpers

Five tree navigation methods (`getFeatureFromId`, `getChildFeatureIds`, `generateNewIds`, `addChild`, `findAndDeleteChildFeature`) are specific to the nested document model and are no longer needed with flat rows. The MongoDB backend retains its own tree traversal via `MongoFeatureRepository`.

Removed dead code

- `backendPostValidate()` -- never called from the active code path. Entire validation layer (`ParentChildValidation`, `ValidationSet`) removed.
- `LocalGFF3DataStore` / `executeOnLocalGFF3` -- never implemented (all throw "not implemented"). Removed from all Operations and Changes.
- `allIds` on `AddFeatureChangeDetails` -- carried forward as dead weight in the serialized change format. Removed from interface and all client code.
- `@apollo-annotation/schemas` dependency -- Mongoose schema types removed from `apollo-common` and `apollo-shared`.

Potential Optimizations

R-tree spatial indexes

Current B-tree on `(refSeq, min, max)` narrows on one bound at a time. R-tree indexes (SQLite native, PostgreSQL GiST) prune on both bounds simultaneously -- useful for dense feature regions.

Bulk import via COPY

PostgreSQL's `COPY FROM` inserts millions of rows/second from flat files. A GFF3 -> CSV -> COPY pipeline would be faster than ORM-level insertion.

Explicit migrations

Currently using `SchemaGenerator.updateSchema()` at startup. Transitioning to committed migration files would provide auditable, reversible schema evolution -- restoring the discipline Apollo2 had with Liquibase.

NestJS Code Issues Found

Issue	Location	Fix
No DTO validation (invalid data reaches service layer)	7 DTO files	Add
ChangesService has too many responsibilities	`changes.service.ts`	Extra
Duplicated ServerDataStore factory	`changes.service.ts`, `operations.service.ts`	Extra
Silent auth failures (generic 403)	`validation.guards.ts`	Throw
Duplicate OAuth guards	`google.guard.ts`, `microsoft.guard.ts`	Gene
Inefficient admin check on login	`authentication.service.ts`	`count`

Security issues (OAuth file-read bug, open redirect, etc.) are tracked in the *Authentication & Security Audit* section.

Per-Gene History Tracking

Background

On origin/main, Apollo3's change log is a **global stream** -- every edit is recorded, but there is no way to query "all edits to gene X" without scanning the entire change table. Apollo2 had per-gene history via Grails audit logging.

The `changedIds` field stores affected feature IDs as a JSON blob. JSON arrays can't be efficiently indexed in SQLite, and PostgreSQL JSONB GIN indexes require database-specific syntax that breaks cross-DB portability.

The Fix: `geneId` Column (Implemented)

An indexed `geneId` column was added to `ChangeEntity`. Each feature change records its top-level gene's ID (resolved via `findRootParentsOfMany`).

```
SELECT * FROM change WHERE gene_id = ? ORDER BY sequence DESC
```

Standard indexed lookup, works identically in SQLite and PostgreSQL, O(log n).

Comparison

Query	Apollo2	Apollo3 (origin/main)	Apollo3 (proposed)
History of gene X	Direct audit table query	Full table scan of JSON	Indexed `WHERE geneId = ?`
Changes by user Y	Scan	Indexed	Same
Changes since time T	Scan	Indexed by `sequence`	Same
Undo support	Limited (view-only)	Full (`getInverse()`)	Same

What was implemented

Phase 1 -- Backend + UI (done):

1. Nullable indexed `geneId` column on `ChangeEntity`
2. `ChangesService.create()` resolves the top-level gene via `findRootParentsOfMany()` and stores its `_id` as `geneId`
3. `GET /changes/gene/:geneId?limit=&page=` -- paginated per-gene history with total count
4. `GET /changes?geneId=<id>` -- the generic findAll endpoint also supports `geneId` filtering
5. The Recent Changes page (`/ui/changes/`) has a search box to look up history by gene/feature ID, with deep-link support via `?geneId=<id>`

"Backfilling" from existing Apollo 3 MongoDB data

For databases with existing change records, a backfill script would walk the change table and populate the `geneId` column retroactively.

Apollo 2 Data Migration

The problem with GFF3-only migration

Exporting GFF3 from Apollo2 and importing into Apollo3 preserves annotations but **loses all edit history** -- who created/modified each gene, when, and what previous states were.

Direct history migration

Map Apollo2 audit records to Apollo3 `ChangeEntity` rows:

Apollo2 field	Apollo3 mapping
Audit operation (INSERT/UPDATE/DELETE)	`typeName` (AddFeature/*/DeleteFeature)
User	`user`
Timestamp	`createdAt` (preserved)
Property changes	`changes` (JSON)
Feature -> organism/sequence	`assembly`
Feature -> top-level gene	`geneId`

Imported records use an `Apollo2Import:` prefix on `typeName` -- viewable but **not undoable** (no `getInverse()` implementation).

Combined workflow: Apollo 2 -> Apollo 3

1. Export annotations from Apollo2 as GFF3
2. Run history migration script against Apollo2 PostgreSQL
3. Import GFF3 into Apollo3
4. Verify per-gene history shows unified timeline